

Causal Language Models

Meng Jiang¹

¹Department of Computer Science and Engineering (CSE)
University of Notre Dame

CSE 60556 LLM

Table of Contents

- 1 GPT-1 & GPT-2
- 2 Practice with 'distilgpt2'
- 3 Insights

Table of Contents

1 GPT-1 & GPT-2

2 Practice with 'distilgpt2'

3 Insights

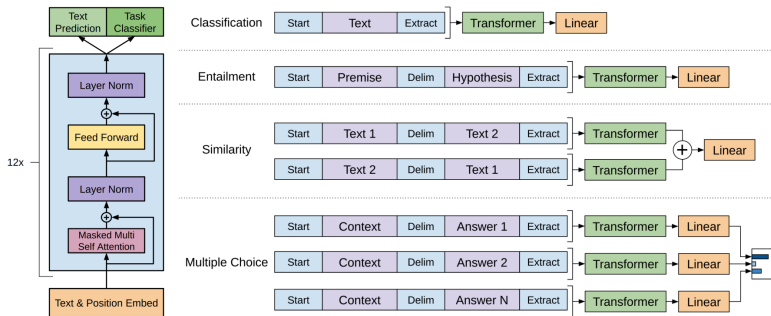
Timeline

- 06/2017: Transformer
- 06/2018: GPT-1
- 10/2018: BERT
- 02/2019: GPT-2
- 10/2019: BART, T5
- 05/2020: GPT-3
- ...

GPT-1

Pre-trained on predicting next words (in a unidirectional, left-to-right, way.) – naturally autoregressive language modeling.

Supervised fine-tuning: add linear layers.



from Radford et al. 2017: Improving Language Understanding by Generative Pre-Training

GPT-1: Fine-tuning Tasks

- Text generation:
*In the midnight, the moon $\Rightarrow$ **shone brightly, illuminating the quiet streets.***
- Text classification:
*The weather today is sunny. $\Rightarrow$ **positive**, negative, neutral*
- Text entailment:
Premise: A soccer game with multiple males playing.
Hypothesis: Some men are playing a sport.
 $\Rightarrow$ **entailment**, contradiction, neutral
- Text similarity:
A: The car is fast and red.
B: The vehicle is quick and crimson.
 $\Rightarrow$ **0.85** $\in [0, 1]$
- Multi-choice QA: ...

GPT-1 vs BERT

GPT-1 uses a stack of **unidirectional decoders** for text generation.
BERT uses a stack of **bidirectional encoders** for text understanding.

	GPT-1	BERT-base	BERT-large
Parameters	117 million	110 million	340 million
Layers	12	12	24
Context token size	512	512	512
Hidden dimensions	768	768	1024
Batch size	64	256	256

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

BERT is good at **language understanding** tasks. GPT is better suited for **language generation** tasks.

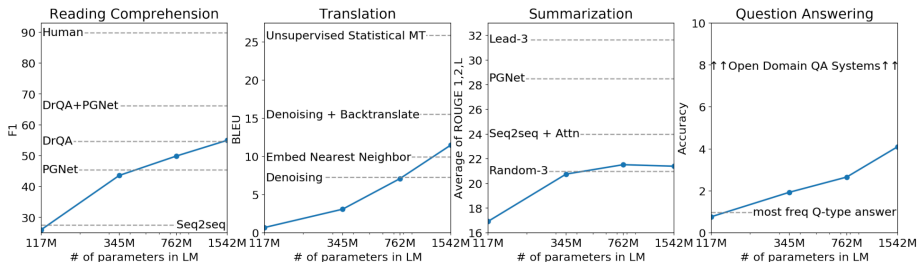
GPT-2 is a direct scale-up of GPT-1, 10 times larger. It can carry out multiple tasks due to (a) larger and cleaner training data and (b) larger number of parameters.

	GPT-1	GPT-2
Parameters	117 million	1.5 billion
Decoder layers	12	48
Context token size	512	1024
Hidden dimensions	768	1600
Batch size	64	512

from Radford et al. 2019: Language Models are Unsupervised Multitask Learners

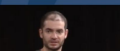
GPT-2: Results

Zero-shot task performance of WebText LMs as a function of model size on many NLP tasks:



GPT-2: Scaling Law started to emerge

ScaledML Conference
Matroid
March 27-28, 2019



GPT-2: Big Improvement Across the Board

Dataset	Metric	Our Result	Previous Record	Human
Winograd Schema Challenge	accuracy (%)	70.70%	63.7%	93%
LAMBADA	accuracy (%)	62.24%	53.23%	95%
LAMBADA	perplexity (↓)	8.6	99	-1-2
Children's Book Test (Common Nouns validation accuracy)	accuracy (%)	93.36%	85.7%	96%
Children's Book Test (Common Nouns validation accuracy)	accuracy (%)	88.06%	82.2%	92%
Page Title Bank	perplexity (↓)	26.7%	45.24	unknown
WikiText-2	perplexity (↓)	18.34	29.34	unknown
jeopardy	bits per character (↓)	0.93	0.98	unknown
test8	bits per character (↓)	0.96	1.08	unknown
WikiText-103	perplexity (↓)	17.48	18.3	unknown

GPT-2 achieves state-of-the-art on Winograd Schema, LAMBADA, and other language modeling tasks.

	LAMBADA (PPL)	LAMBADA (ACC)	CBT-CN (ACC)	CBT-NE (ACC)	WikiText2 (PPL)	PTB (PPL)	enwik8 (BPB)	text8 (BPC)	WikiText103 (PPL)	1BW (PPL)
SOTA	99.8	59.23	85.7	82.3	39.14	46.54	0.99	1.08	18.3	21.8
117M	35.13	45.99	87.65	83.4	29.41	65.85	1.16	1.17	37.50	75.20
345M	15.60	55.48	92.35	87.1	22.76	47.33	1.01	1.06	26.37	55.72
762M	10.87	60.12	93.45	88.0	19.93	40.31	0.97	1.02	22.05	44.575
1542M	8.63	63.24	93.30	89.05	18.34	35.76	0.93	0.98	17.48	42.16

Table of Contents

1 GPT-1 & GPT-2

2 Practice with 'distilgpt2'

3 Insights

Dive into Transformers with Hugging Face

```
gpt_tokenizer = AutoTokenizer.from_pretrained(LOCAL_CAUSAL_LM)
gpt_model = AutoModelForCausalLM.from_pretrained(LOCAL_CAUSAL_LM,
    output_attentions=True,
    output_hidden_states=True).to(DEVICE)
gpt_model.eval()

text = "Transformers in Hugging Face make it easy to reuse
    pretrained language models."
inputs = gpt_tokenizer(text, return_tensors="pt")

with torch.no_grad():
    outputs = gpt_model(**inputs)

print(outputs.keys())
```

Output:

```
odict_keys(['logits', 'past_key_values', 'hidden_states', '
    attentions'])
```

Architecture Code

modeling_distilbert.py: 'distilgpt2' as one of Transformer models

```
class DistilBertSelfAttention(nn.Module):
    def __init__(self, config: PreTrainedConfig):
        super().__init__()
        self.config = config

        self.n_heads = config.n_heads
        self.dim = config.dim
        self.attention_head_size = self.dim // self.n_heads
        self.scaling = self.attention_head_size**-0.5

        self.q_lin = nn.Linear(in_features=config.dim,
                                out_features=config.dim)
        self.k_lin = ...
        self.v_lin = ...
        self.out_lin = ...

        self.dropout = nn.Dropout(p=config.attention_dropout)
```

Architecture Code

```
def forward(self, hidden_states, attention_mask ...):
    input_shape = hidden_states.shape[:-1]
    hidden_shape = (*input_shape, -1, self.attention_head_size)
    query_layer = self.q_lin(hidden_states).view(*hidden_shape).
        transpose(1, 2)
    key_layer = self.k_lin(hidden_states).view(*hidden_shape).
        transpose(1, 2)
    value_layer = self.v_lin(hidden_states).view(*hidden_shape).
        transpose(1, 2)
    attention_interface: Callable = ALL_ATTENTION_FUNCTIONS.
        get_interface(
            self.config._attn_implementation, eager_attention_forward
        )
    attn_output, attn_weights = attention_interface(
        self, query_layer, key_layer, value_layer, attention_mask,
        ...
    ) ...
```

Architecture Code

```
def eager_attention_forward(module, query, key, value,
    attention_mask, scaling, dropout ...):
    if scaling is None:
        scaling = query.size(-1) ** -0.5
    # Take the dot product between "query" and "key" to get the
    # raw attention scores.
    attn_weights = torch.matmul(query, key.transpose(2, 3)) *
        scaling
    if attention_mask is not None:
        attn_weights = attn_weights + attention_mask
    attn_weights = nn.functional.softmax(attn_weights, dim=-1)
    attn_weights = nn.functional.dropout(attn_weights, p=dropout,
        training=module.training)
    attn_output = torch.matmul(attn_weights, value)
    attn_output = attn_output.transpose(1, 2).contiguous()
    return attn_output, attn_weights
```

Dive into Transformers with Hugging Face

```
attentions = outputs.attentions
print(len(attentions)) # number of layers
print(attentions[0].shape) # [batch, heads, query_tokens,
    key_tokens]
layer, head = 0, 0
attn = attentions[layer][0, head] # [seq_len, seq_len]
layer_last, head = -1, 0
attn_last = attentions[layer_last][0, head] # [seq_len, seq_len]
tokens = gpt_tokenizer.convert_ids_to_tokens(inputs["input_ids"
    ][0])
print(tokens)
```

Output:

```
6
torch.Size([1, 12, 16, 16])
['Transform', 'ers', 'Ġin', 'ĠHug', 'ging', 'ĠFace', 'Ġmake', 'Ġit', 'Ġeasy', 'Ġto', 'Ġreuse', 'Ġpret', 'rained', 'Ġlanguage', 'Ġmodels', '.']
```


Visualize Attentions

```
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 6))
plt.imshow(attn)
plt.xticks(range(len(tokens)), tokens, rotation=90)
plt.yticks(range(len(tokens)), tokens)
plt.xlabel("Keys / values attended to")
plt.ylabel("Queries")
plt.title(f"Layer {layer}, Head {head} attention")
plt.colorbar()
plt.show()
```

Visualize Attentions

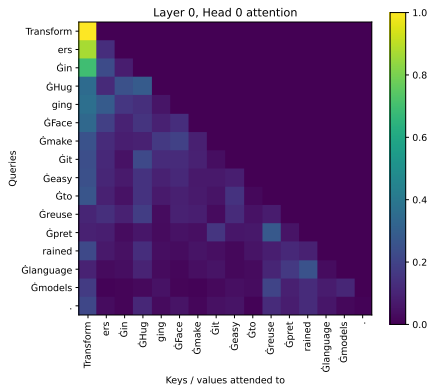


Figure: First layer, first head

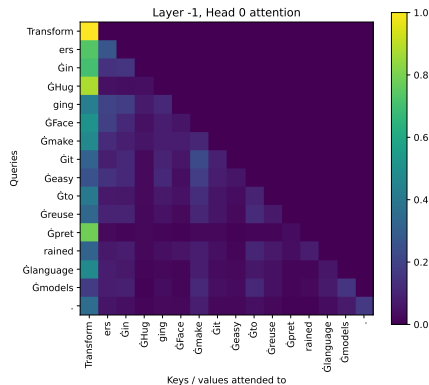


Figure: Last layer, first head

Table of Contents

1 GPT-1 & GPT-2

2 Practice with 'distilgpt2'

3 Insights

Causal attention learns implicit positional encodings.

Conjecture: Causal attention allows models to predict the number of attendable tokens at each position. Such a mechanism could effectively encode the absolute position of each token into its representation.

Observation: GPT does not rely on explicit positional encodings.

Analysis: (1) Some notion of absolute positions exists in the hidden layers of LMs even when they are trained without explicit positional encoding, and that this information is acquired throughout the first few layers. GPTs leverage the causal attention to estimate the number of preceding tokens for each current token, thereby approximating its absolute position.

(2) Bidirectional transformer encoders do not contain causal attention masks or any other limitation on the attention mechanism; thus, they should be unable to learn absolute positions without positional encoding.

from Haviv et al. 2022: Transformer Language Models without Positional Encodings Still Learn Positional Information

Path to zero-shot Generalization?

Question: How do causal attention mechanisms and multi-task learning affect zero-shot capabilities?

Findings: (1) GPT models trained with an autoregressive language modeling objective perform best in zero-shot generalization after [unsupervised pre-training](#).

(2) BERT models trained with masked language modeling achieve the best zero-shot performance after [multitask fine-tuning](#).

from Wang et al. 2022: What Language Model Architecture and Pre-training Objective Work Best for Zero-shot Generalization?

Takeaways

- GPT-1 was pre-trained on predicting next words.
- It added linear layers for downstream tasks.
- GPT-2 was ten times larger, showing zero-shot task performance.
- Causal attention learned implicit positional encodings.
- What architecture would enable superior zero-shot generalization?